06 — Statistics and Estimates

> *"The planner is only as good as its statistics. Bad statistics = bad plans."*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Explain what ANALYZE does and how it samples table data
- Read pg_stats to understand what statistics the planner has available
- Describe how MCV (most common values) and histogram bounds are used
- Configure statistics_target for specific columns
- Create extended statistics for correlated or multi-column predicates
- Detect and resolve stale statistics causing bad plans

What ANALYZE Does

`ANALYZE` samples a fraction of a table's rows (by default ~30,000 rows via random sampling) and computes statistics that are stored in `pg_statistic`. These statistics are then used by the query planner to estimate selectivity.

```
-- Analyze entire database
ANALYZE;

-- Analyze specific table
ANALYZE orders;
```

```
-- Analyze specific columns (faster if table is large)
ANALYZE orders(customer_id, status, created_at);

-- With verbose output (shows what's happening)
ANALYZE VERBOSE orders;
```

What ANALYZE Computes

For each column, ANALYZE computes:

1. Null fraction (fraction of NULLs)
2. Average width (bytes)
3. Number of distinct values
4. Most common values + their frequencies
5. Histogram of value distribution
6. Physical-to-logical correlation

Sampling

ANALYZE doesn't read the entire table — it samples approximately `300 × statistics_target` pages (default: 300 × 100 = 30,000 rows). For very large tables this may not be representative. Increase `statistics_target` for better accuracy.

---

pg_statistic Internals

`pg_statistic` stores statistics as arrays with a complex encoding. Most users access it via the `pg_stats` view.

```
-- Raw pg_statistic (complex, avoid direct access)
SELECT * FROM pg_statistic WHERE starelid = 'orders'::regclass;

-- More accessible view:
SELECT * FROM pg_stats WHERE tablename = 'orders';
```

Key columns in pg_statistic:

- `stakind1..5` : type of statistic stored (1=MCV, 2=histogram, 3=correlation, 4=MCV for elements, 5=histogram of distinct counts)
- `staop1..5` : operator used for comparison
- `stanumbers1..5` : numeric statistics (frequencies, correlation)
- `stavalues1..5` : value statistics (MCVs, histogram bounds)

---

pg_stats: Human-Readable View

```
SELECT
    tablename,
    attname,
    null_frac,
    avg_width,
```

```
    n_distinct,
    most_common_vals,
    most_common_freqs,
    histogram_bounds,
    correlation
FROM pg_stats
WHERE tablename = 'orders'
ORDER BY attname;
```

Column Descriptions

Column	Type	Description
`null_frac`	float4	Fraction of NULLs (0.0 = no nulls)
`avg_width`	int4	Average width in bytes
`n_distinct`	float4	If >0: distinct count. If <0: fraction of total rows
`most_common_vals`	anyarray	Up to `statistics_target` most common values
`most_common_freqs`	float4[]	Corresponding frequencies
`histogram_bounds`	anyarray	Boundaries of equal-frequency histogram buckets
`correlation`	float4	Physical vs logical ordering correlation (-1 to 1)

n_distinct Interpretation

```
n_distinct = 200     → exactly 200 distinct values (estimated)
n_distinct = -0.001  → 0.1% of rows have unique values (e.g., 1000 distinct in 1M row table)
n_distinct = -1.0    → every row has a unique value (high-cardinality column like PK)
```

ASCII Diagram: How Statistics Drive Estimates

```
Table: orders (1,000,000 rows)
Column: status TEXT

pg_stats for status:
  most_common_vals:  {completed, pending, processing}
  most_common_freqs: {0.85,      0.10,    0.05}

Query: SELECT COUNT(*) FROM orders WHERE status = 'pending';
  Planner looks up 'pending' in most_common_vals
  Finds at index 1 → frequency = 0.10
  Estimated rows = 1,000,000 × 0.10 = 100,000
  ✓ Accurate: actual ~= 100,000

Query: SELECT COUNT(*) FROM orders WHERE status = 'cancelled';
  'cancelled' NOT in most_common_vals
```

```
  Estimated: (1.0 - 0.85 - 0.10 - 0.05) / (n_distinct - 3) = 0 / 0 = ~0.005%
  Estimated rows = 1,000,000 × 0.00005 = 50
  ✗ If 'cancelled' actually has 50,000 rows: huge underestimate!
  Fix: ANALYZE (will add 'cancelled' to MCV if common enough)
       Or: increase statistics_target to capture more MCVs

Column: amount NUMERIC

pg_stats for amount:
  histogram_bounds: {0.5, 10.2, 25.8, 50.1, 100.5, 250.3, 500.2, 1000.8, 9999.9}
  (9 bounds = 8 equal-frequency buckets, each ~= 125,000 rows)

Query: SELECT * FROM orders WHERE amount BETWEEN 100 AND 250;
  Find bucket containing 100: bucket 5 (100.5 bound) → ~= 12.5% of rows
  Find bucket containing 250: bucket 6 (250.3 bound) → ~= 12.5% of rows
  Estimate: ~25% of rows in range = 250,000
  Actual: depends on distribution within buckets

  If we increase statistics_target from 100 to 500:
    More histogram buckets → more accurate range estimates
    histogram_bounds has 500 entries instead of 100
    Better precision for range queries
```

Most Common Values (MCV)

MCVs store the most frequently occurring values for a column.

How MCV Works for Equality Queries

```
most_common_vals:  {completed, pending, processing, cancelled}
most_common_freqs: {0.85,      0.10,    0.03,       0.01}

WHERE status = 'completed' → use freq[0] = 0.85 → 850,000 rows
WHERE status = 'pending'   → use freq[1] = 0.10 → 100,000 rows
WHERE status = 'unknown'   → not in MCV, use: (1 - sum(freqs)) / (n_distinct -
MCV_count)
                           → (1 - 0.99) / (4 - 4) = 0 / 0 → ~0.005% default
```

Low-Cardinality Columns

For columns with few distinct values (e.g., boolean, status, type), MCVs cover all or most distinct values accurately.

High-Cardinality Columns

For columns with millions of distinct values (customer_id, email), MCVs only capture the most frequent values. Histogram bounds handle the rest.

Histogram Bounds

Histograms store the distribution of values for range predicate estimation.

Equal-Frequency Histogram

PostgreSQL uses an equal-frequency (equi-depth) histogram: each bucket contains approximately the same number of rows.

```
histogram_bounds: {10, 25, 50, 100, 250, 500, 1000}
6 bounds = 5 buckets, each ~= 200,000 rows (for 1M total rows)

Bucket 1: [10,   25)  → 200,000 rows
Bucket 2: [25,   50)  → 200,000 rows
Bucket 3: [50,  100)  → 200,000 rows
Bucket 4: [100, 250)  → 200,000 rows
Bucket 5: [250, 500)  → 200,000 rows
           remainder  → (1000+) remaining rows not in histogram

Query: WHERE amount BETWEEN 75 AND 150
  Bucket 3: amount 75-100 covers [75,100) / [50,100) = 50% of bucket = 100,000
  Bucket 4: amount 100-150 covers [100,150) / [100,250) = 33% of bucket = 66,667
  Total estimate: 166,667 rows
```

More Buckets = More Accurate Range Estimates

The default 100 buckets works well for most columns. For highly skewed distributions or critical range queries, increase the statistics_target.

Correlation Statistic

`correlation` measures how well a column's logical sort order matches the physical (heap) order of rows.

```
correlation = 1.0:  rows are physically stored in ascending column order
              0.0:  no relationship between logical and physical order
             -1.0:  rows are physically stored in descending column order
```

How Correlation Affects Plans

For Index Scans: High correlation → rows with similar values are on the same heap pages → index scan reads fewer heap pages (consecutive in heap = good locality).

Low correlation → rows with similar values are scattered across heap → index scan reads many random heap pages → planner may prefer Seq Scan.

For BRIN Indexes: BRIN only works when correlation is high (see [../07_Indexes/06_brin_indexes.md]).

```sql
-- Check correlation for all columns in a table
SELECT attname, correlation
FROM pg_stats
WHERE tablename = 'orders'
ORDER BY abs(correlation) DESC;

-- Output:
-- id         |  0.998  ← BIGSERIAL, always inserted in order
-- created_at |  0.993  ← insert timestamp, highly correlated
```

```
-- status      |  0.002  ← updated randomly, very low correlation
-- customer_id| -0.012  ← random assignment, very low correlation
```

---

statistics_target Configuration

`statistics_target` controls how many values are sampled for MCV and histogram.

Default (100)

- `most_common_vals` : up to 100 entries
- `histogram_bounds` : up to 100 buckets
- Samples ~30,000 rows from the table

When to Increase

1. **Column appears in WHERE predicates** with high variance in selectivity
2. **Range queries** that are frequently off in estimates
3. **High-cardinality columns** where the top 100 MCVs don't represent the distribution

```
-- Increase for specific columns
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 200;

-- Increase globally (applies to all future ANALYZE calls)
ALTER SYSTEM SET default_statistics_target = 200;
SELECT pg_reload_conf();

-- Apply the new statistics_target:
ANALYZE orders;
ANALYZE events;
```

When to Decrease

For columns never used in WHERE predicates, statistics collection is wasted effort.

```
ALTER TABLE orders ALTER COLUMN internal_notes SET STATISTICS 0;  -- don't collect
ANALYZE orders;
```

Extended Statistics

PostgreSQL 10+ supports **extended statistics** for capturing multi-column correlations.

Types of Extended Statistics

1. **ndistinct**: distinct values for combinations of columns
2. **dependencies**: functional dependencies between columns (if A determines B)
3. **mcv**: most common value combinations for groups of columns

```
-- Create extended statistics
CREATE STATISTICS stats_orders_status_region
```

```
    ON status, region
    FROM orders;

-- Specify what to collect (default: all applicable types)
CREATE STATISTICS stats_city_state (ndistinct, dependencies, mcv)
    ON city, state
    FROM addresses;

-- Run ANALYZE to populate the statistics
ANALYZE orders;
ANALYZE addresses;
```

Checking Extended Statistics

```
-- View extended statistics definitions
SELECT * FROM pg_statistic_ext WHERE starelid = 'orders'::regclass;

-- View actual statistics data
SELECT * FROM pg_statistic_ext_data
WHERE stxoid = 'stats_orders_status_region'::regclass;
```

When Extended Statistics Help

```
-- Without extended statistics:
-- WHERE status = 'pending' AND region = 'US'
-- Estimate = P(pending) × P(US) = 0.10 × 0.30 = 3%
-- If actually these are correlated (pending orders skew to US): wrong!

-- With CREATE STATISTICS ON status, region:
-- Planner uses joint distribution → accurate estimate
-- Better plan choice (correct join algorithm, index usage)
```

Functional Dependencies

```
-- zip_code → city (zip uniquely determines city)
CREATE STATISTICS stats_zip_city (dependencies) ON zip_code, city FROM addresses;
ANALYZE addresses;

-- Without: P(zip='10001' AND city='New York') = P(zip) × P(city) (wrong, double-
counting)
-- With: planner knows city is determined by zip, uses P(zip) alone (correct)
```

Table-Level and Column-Level ANALYZE

```
-- Analyze all columns of a table
ANALYZE orders;
```

```
-- Analyze only specific columns (faster for large tables)
ANALYZE orders(customer_id, status, created_at);

-- Analyze all tables in a schema
ANALYZE public.*;  -- PostgreSQL 14+ syntax
-- or:
DO $$
DECLARE r record;
BEGIN
    FOR r IN SELECT tablename FROM pg_tables WHERE schemaname = 'public' LOOP
        EXECUTE 'ANALYZE ' || quote_ident(r.tablename);
    END LOOP;
END $$;

-- VACUUM ANALYZE (recommended: do both in one pass)
VACUUM ANALYZE orders;
```

Autovacuum and Statistics

Autovacuum automatically runs ANALYZE when:

```
rows_changed_since_last_analyze >=
    autovacuum_analyze_threshold + autovacuum_analyze_scale_factor × reltuples
```

Default settings:

```
SHOW autovacuum_analyze_threshold;    -- default 50
SHOW autovacuum_analyze_scale_factor; -- default 0.2 (20% of table)
```

For a 1M row table: triggers after 50 + 0.2 × 1,000,000 = 200,050 rows changed.

Tuning Autovacuum for Statistics

```
-- More frequent analysis for critical large tables:
ALTER TABLE orders SET (
    autovacuum_analyze_scale_factor = 0.01,  -- analyze after 1% changes
    autovacuum_analyze_threshold = 1000      -- minimum 1000 changed rows
);
```

Detecting Stale Statistics

Signs of Stale Statistics

1. `EXPLAIN ANALYZE` shows `actual rows` >> `estimated rows` (10× or more)
2. Queries suddenly became much slower after a bulk load
3. `last_analyze` is very old relative to `n_mod_since_analyze`

```sql
-- Find tables with stale statistics
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_mod_since_analyze,
    last_analyze,
    last_autoanalyze,
    ROUND(100.0 * n_mod_since_analyze / NULLIF(n_live_tup, 0), 1) AS mod_pct
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 10000
  OR (last_analyze IS NULL AND n_live_tup > 0)
ORDER BY n_mod_since_analyze DESC
LIMIT 20;
```

Quick Fix

```sql
-- After bulk operations:
ANALYZE important_table;
-- or more thorough:
VACUUM ANALYZE important_table;
```

Common Mistakes

1. Not running ANALYZE after bulk loads

```sql
-- After COPY or large INSERT:
COPY orders FROM '/path/to/data.csv';
-- Without ANALYZE: planner uses old statistics (before the load)
-- Fix: immediately after:
ANALYZE orders;
```

2. Using default statistics_target for heavily-queried columns

```sql
-- Column used in complex range predicates:
ALTER TABLE events ALTER COLUMN event_time SET STATISTICS 500;
ANALYZE events;
-- Better histogram resolution → more accurate range estimates
```

3. Forgetting that ANALYZE is statistical (sampling)

ANALYZE samples ~30,000 rows by default. For tables with 1 billion rows, this is only 0.003% — potentially missing rare values. Increase statistics_target for rare-value columns.

4. Not using extended statistics for correlated columns

If you see consistently wrong row estimates for multi-column predicates, check if the columns are correlated and create extended statistics.

5. Disabling autovacuum without alternative maintenance

```
-- NEVER just do this without a replacement plan:
ALTER TABLE orders SET (autovacuum_enabled = false);
-- If you must disable it, schedule manual VACUUM ANALYZE
```

Best Practices

1. **Always ANALYZE after bulk loads** — don't wait for autovacuum.
2. **Increase statistics_target for query-critical columns**:

   ```
   ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
   ```

3. **Create extended statistics for correlated column pairs** commonly used together in WHERE clauses.
4. **Monitor** `pg_stat_user_tables.n_mod_since_analyze` — alert when > 10% of rows modified since last analyze.
5. **Tune autovacuum scale factors** for large, frequently-updated tables to ensure statistics remain current.
6. **Use** `EXPLAIN (ANALYZE, BUFFERS)` whenever investigating slow queries — check actual vs estimated rows.

Performance Considerations

ANALYZE Cost

ANALYZE reads a sample of the table (not the full table), so it's fast even for very large tables. However, for tables with millions of pages, even sampling can take seconds. Schedule ANALYZE during off-peak hours for very large tables.

pg_statistic Cache

Statistics are read from pg_statistic at plan time and may be cached in the planner. After running ANALYZE, new statistics are available immediately for new query plans.

Interview Questions & Answers

Q1. What does ANALYZE do and when should you run it?

A: ANALYZE reads a sample of a table (default ~30,000 rows) and computes per-column statistics: null fraction, average width, distinct value count, most common values with frequencies, histogram bounds, and physical ordering correlation. These statistics are stored in pg_statistic and used by the query planner to estimate selectivity. You should run ANALYZE: (1) Immediately after bulk data loads (COPY, large INSERT). (2) After large UPDATE or DELETE operations that significantly change data distribution. (3) Whenever EXPLAIN ANALYZE

shows actual rows far from estimated rows. Autovacuum handles routine ANALYZE, but manual analysis is needed after large one-time operations.

Q2. What is a histogram bound and how does it help with range queries?

A: Histogram bounds define the boundaries of equal-frequency buckets that represent the column's value distribution. PostgreSQL creates an equi-depth histogram (each bucket contains approximately the same number of rows). For a range query `WHERE amount BETWEEN 100 AND 500`, the planner finds which histogram buckets overlap the range and estimates the fraction of rows they cover. More buckets (higher statistics_target) mean more precise range estimates. The default 100 buckets works for most cases, but for columns with complex or skewed distributions queried with range predicates, increasing to 200-500 can significantly improve estimate accuracy.

Q3. What are extended statistics and when do you need them?

A: Extended statistics (CREATE STATISTICS) capture multi-column correlations that the standard per-column statistics miss. Types: (1) `dependencies` — captures functional dependencies (A → B means knowing A determines B). (2) `ndistinct` — captures distinct value counts for column combinations. (3) `mcv` — captures most common value combinations. You need extended statistics when queries filter on multiple correlated columns and EXPLAIN ANALYZE shows consistently wrong row estimates. For example, if `city` and `zip_code` are correlated, the planner's independence assumption (`P(city) × P(zip)`) underestimates selectivity. Extended statistics teach the planner the actual joint distribution.

Exercises with Solutions

Exercise 1

A query `WHERE status = 'pending' AND created_at > '2024-01-01'` has estimated rows=5 but actual rows=50,000. Diagnose and fix.

Solution:

```
-- Step 1: Check statistics for both columns
SELECT attname, null_frac, n_distinct, most_common_vals, most_common_freqs,
       correlation, pg_stats_last_update
FROM pg_stats
WHERE tablename = 'orders' AND attname IN ('status', 'created_at');

-- Possible findings:
-- A) last_analyze is old → run ANALYZE orders;
-- B) 'pending' not in most_common_vals → data was bulk-inserted recently
-- C) created_at histogram is too coarse → increase statistics_target

-- Step 2: Refresh statistics
ANALYZE orders;

-- Step 3: If still wrong, check correlation between columns
-- Maybe orders often transition to 'pending' recently?
CREATE STATISTICS stats_orders_status_date (dependencies, mcv)
    ON status, created_at FROM orders;
ANALYZE orders;
```

```
-- Step 4: Verify
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE status = 'pending' AND created_at > '2024-01-01';
-- Check: estimated rows should now be close to actual 50,000
```

Production Scenarios

Scenario 1: Planner Underestimates by 100x After Migration

After migrating 50M orders from MySQL, all PostgreSQL queries were using Nested Loop joins that were catastrophically slow for large datasets.

Root cause: The table had statistics from before the migration (100K rows). The 50M new rows were loaded via COPY without running ANALYZE.

```
-- After bulk load, ALWAYS run:
VACUUM ANALYZE orders;

-- Verify:
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Before: rows estimate=1000, actual=5,000,000 (5000x off)
-- After ANALYZE: rows estimate=5,100,000, actual=5,000,000 (2% off)
-- Query now uses Hash Aggregate instead of Nested Loop: 45s → 3s
```

Cross-References

- 05_query_planner.md — How statistics are used in planning
- 02_explain_analyze.md — Spotting estimate problems in EXPLAIN
- ../07_Indexes/06_brin_indexes.md — Correlation and BRIN
- 09_slow_query_analysis.md — Finding slow queries